Day 112 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 1 reference

Watch: Support Vector Machines Part 1 (of 3): Main Ideas!!! (StatQuest) | Support Vector Machines in Python from Start to Finish (StatQuest)

`day112_support_vector_machines.py`, segment by segment, in the order `main()` uses things. The solver is NumPy; scikit-learn supplies the datasets and SVC as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: w, b = the boundary · α = one multiplier per point · C = cost of a violation · K = the matrix of dot products · f(x) = w·x + b

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, "..", "week-15-practical-nlp-pretrained"))
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
CS = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
SPLITS = 20
TOL = 1e-3
```

- `import day99_exercise as d99` — Day 99's bootstrap interval, for the comparisons over splits.
- `CS = [0.001, ..., 1000]` — C on a log scale: each value ten times the last.
- `TOL = 1e-3` — SMO stops when the worst violation is smaller than this.

Segment 1 — The solver

smo: two multipliers at a time

MATH

$a_j \leftarrow a_j + y_j(E_i - E_j) / \eta, \quad \eta = K_{ii} + K_{jj} - 2K_{ij}$, *then clip;* a_i *moves to keep* $\Sigma a\, y = 0$

CODE

```
def smo(K, y, C, tol=TOL, max_iter=200000):
    n = len(y); a = np.zeros(n); f = np.zeros(n)
    for it in range(max_iter):
        G = y * f - 1
        up = ((a < C) & (y > 0)) | ((a > 0) & (y < 0))
        low = ((a < C) & (y < 0)) | ((a > 0) & (y > 0))
        v = -y * G
        i = np.where(up)[0][np.argmax(v[up])]; j = np.where(low)[0][np.argmin(v[low])]
        if v[i] - v[j] < tol:
            break
        eta = max(K[i, i] + K[j, j] - 2 * K[i, j], 1e-12)
        if y[i] != y[j]:
            L, H = max(0.0, a[j] - a[i]), min(C, C + a[j] - a[i])
        else:
            L, H = max(0.0, a[i] + a[j] - C), min(C, a[i] + a[j])
        Ei, Ej = f[i] - y[i], f[j] - y[j]
        aj = np.clip(a[j] + y[j] * (Ei - Ej) / eta, L, H)
        ai = a[i] + y[i] * y[j] * (a[j] - aj)
        ai, aj = [0.0 if v < 1e-12 * C else C if v > C * (1 - 1e-12) else v for v in (ai, aj)]
        f +=(ai - a[i]) * y[i] * K[:, i] + (aj - a[j]) * y[j] * K[:, j]
        a[i], a[j] = ai, aj
    return a, it + 1
```

- `up = ((a < C) & (y > 0)) | ((a > 0) & (y < 0))` — boolean masks with & and |: which multipliers may still move in each direction.
- `np.where(up)[0][np.argmax(v[up])]` — the position, in the whole array, of the largest value within the mask.
- `np.clip(..., L, H)` — keeps the new value inside the box allowed for the pair.
- `[0.0 if v < ... else C if v > ... else v for v in (ai, aj)]` — a chained conditional in a list: snaps rounding errors onto the bounds (the fix that made SMO finish).
- `f += ... * K[:, i] + ... * K[:, j]` — updates every point's output using only two columns of K.

bias and fit_svm

MATH

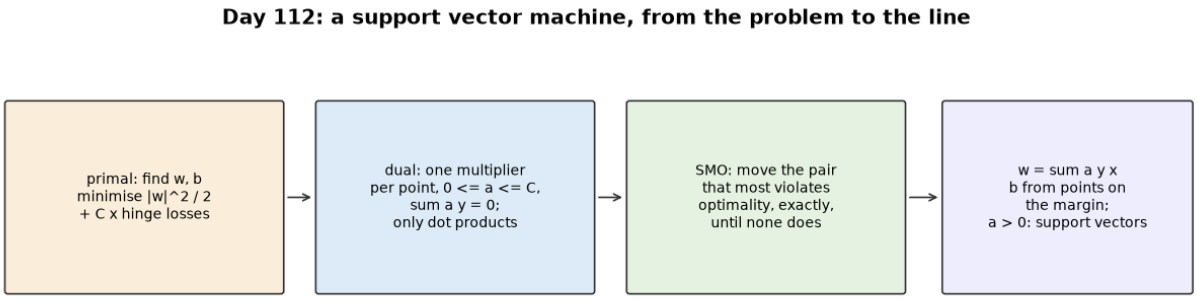
$w = \Sigma \alpha_i y_i x_i; \quad b = y_i - w \cdot x_i$ *for a point on the margin*

CODE

```
def bias(a, y, K, C):
    f = K @ (a * y); e = 1e-8 * max(C, 1)
    free = (a > e) & (a < C - e)
    if free.any():
        return float(np.mean(y[free] - f[free]))
    lo = (a <= e) & (y > 0) | (a >= C - e) & (y < 0)
    return float((np.max((y - f)[lo]) + np.min((y - f)[~lo])) / 2)
def fit_svm(X, y, C):
    K = X @ X.T
    a, _ = smo(K, y, C)
    return (a * y) @ X, bias(a, y, K, C), a
```

- `free = (a > e) & (a < C - e)` — points strictly between the bounds lie exactly on the margin, so y f(x) = 1 gives b.
- `np.max(...[lo]) + np.min(...[~lo])` — with no such point, b may lie anywhere in a range: the middle is taken.
- `(a * y) @ X` — w as a weighted sum of training points; most weights are 0.

SYSTEM DIAGRAM



From the problem to the line.

Segment 2 — Checks and baselines

primal and dual: the two objectives

CODE

```
def primal(w, b, X, y, C):
    return 0.5 * w @ w + C * np.maximum(0, 1 - y * (X @ w + b)).sum()
def dual(a, y, K):
    return a.sum() - 0.5 * (a * y) @ K @ (a * y)
```

- `np.maximum(0, 1 - y * (X @ w + b))` — the hinge loss of every point at once.
- `(a * y) @ K @ (a * y)` — a quadratic form: one number from a vector and a matrix.

perceptron and logistic

CODE

```
def perceptron(X, y, seed, epochs=100):
    rng = np.random.RandomState(seed); w = np.zeros(X.shape[1]); b = 0.0
    for _ in range(epochs):
        mistakes = 0
        for i in rng.permutation(len(y)):
            if y[i] * (X[i] @ w + b) <= 0:
                w += y[i] * X[i]; b += y[i]; mistakes += 1
        if mistakes == 0:
            break
    return w, b
...
def logistic(X, y, lam, steps=3000, lr=0.5):
    w = np.zeros(X.shape[1]); b = 0.0; t = (y > 0).astype(float)
    for _ in range(steps):
        p = 1 / (1 + np.exp(-(X @ w + b)))
        w -= lr * (X.T @ (p - t) / len(y) + lam * w); b -= lr * np.mean(p - t)
    return w, b
```

- `for i in rng.permutation(len(y)):` — a new random order each pass: why the perceptron's answer varies.
- `if y[i] * (X[i] @ w + b) <= 0:` — only mistakes move the line.
- `lam * w` — the L2 penalty's gradient: the same strength as the SVM's C, as 1 / (C n).

Segment 3 — The experiments

kfold_svm_accuracy: scaling inside each fold

CODE

```
def kfold_svm_accuracy(X, y, C, k=5, seed=0):
    folds = np.array_split(np.random.RandomState(seed).permutation(len(y)), k); acc = []
    for i in range(k):
        va = folds[i]; tr = np.concatenate([folds[j] for j in range(k) if j != i])
        A, B = standardise(X[tr], X[va]); w, b, _ = fit_svm(A, y[tr], C)
        acc.append(np.mean(np.sign(B @ w + b) == y[va]))
    return float(np.mean(acc))
```

- `A, B = standardise(X[tr], X[va])` — scaling is fitted on the fold's training rows only, so validation stays unseen.
- `np.sign(B @ w + b) == y[va]` — the sign of f(x) is the predicted class, -1 or +1.

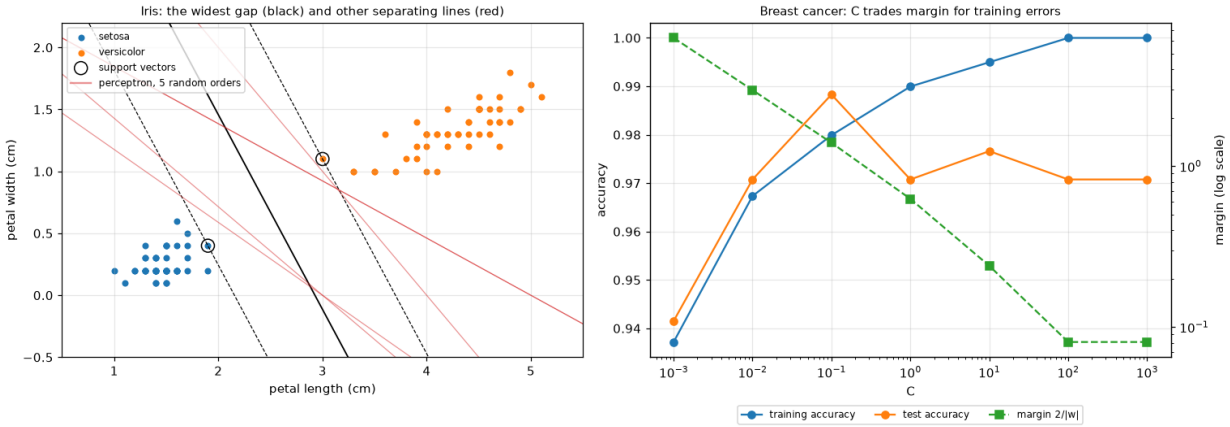
Support vectors only, inside demo_support

CODE

```
w, b, a = fit_svm(A, yt, C); sv = a > 1e-8
w2, b2, _ = fit_svm(A[sv], yt[sv], C)
keep = ~sv; rng = np.random.RandomState(0); drop = rng.choice(np.where(keep)[0], keep.sum() // 2,
    replace=False)
rest = np.setdiff1d(np.arange(len(yt)), drop); w3, b3, _ = fit_svm(A[rest], yt[rest], C)
half = rng.choice(np.where(sv)[0], sv.sum() // 2, replace=False)
rest = np.setdiff1d(np.arange(len(yt)), half); w4, b4, _ = fit_svm(A[rest], yt[rest], C)
```

- `sv = a > 1e-8` — a boolean mask of the support vectors.
- `fit_svm(A[sv], yt[sv], C)` — boolean indexing keeps only the masked rows.
- `np.setdiff1d(np.arange(len(yt)), half)` — every row number except the removed ones.

REAL GRAPH



Real graph from the script: iris margins, and C on breast cancer.

Quick Syntax Glossary — new or reinforced constructs

- `mask_a & mask_b, mask_a | mask_b` — element-wise AND and OR of boolean arrays.
- `np.where(mask)[0][np.argmax(v[mask])]` — the original position of the best masked value.
- `np.clip(x, lo, hi)` — limit a value to a range.
- `v @ M @ v` — a quadratic form.
- `np.sign(z)` — -1, 0 or +1.
- `a if c else b if d else e` — a chained conditional expression.

End of Day 112 syntax reference. For the experiments and what they showed, see the Day 112 complete guide.